



Lokmanya Tilak Jankalyan Shikshan Sanstha's
Lokmanya Tilak College of Engineering
An Autonomous Institute Affiliated to University of Mumbai
(Approved by AICTE, Accredited by NAAC 'A' Grade & four programs by NBA)
Sector-04, Koparkhairane, Navi Mumbai - 400 079



DEPARTMENT OF COMPUTER ENGINEERING

Course Name: Cloud Computing Lab	Course Code: CSL605
Course Project Title: RecipeOS – Cook Smarter, Not Harder	
Student Roll No. & Name: Adarsh pal / TEB230	
Class/Sem/Div: T.E. / VI / B	A.Y.: 2025–26
Date of Submission: 06-04-2026	
Lab Outcomes Covered: LO1 – LO6	
Name and Signature of Faculty with Date	

CONTENTS

❖ Problem Statement

❖ Introduction

❖ Specifications / Requirements

❖ Technology Used

❖ Design

❖ Implementation

❖ Result

❖ References

1. PROBLEM STATEMENT

The modern internet is flooded with recipe blogs packed with lengthy personal stories, irrelevant content, and aggressive advertisements, making it frustrating for users to find and follow recipes efficiently. Existing cooking platforms lack features that support the actual act of cooking — such as real-time step-by-step guidance, simultaneous timer management, and hands-free operation.

Furthermore, users with specific health goals (weight loss, muscle gain, heart health, or plant-based diets) have no intelligent filtering mechanism to discover recipes aligned with their objectives. There is also no gamified skill-progression system that motivates home cooks to improve over time.

The core problems addressed by RecipeOS are:

- Bloated recipe websites with ads, irrelevant life stories, and poor UX.
- No structured, interactive cooking guidance with parallel task timelines.
- Absence of a hands-free 'Cook Mode' for use during active cooking.
- Lack of goal-based recipe filtering tied to health and lifestyle needs.
- No skill-progression or achievement system to motivate beginner cooks.
- No 'Pantry Chef' feature to suggest recipes from available ingredients.
- Recipe platforms are not optimized for cloud deployment and scalability.

RecipeOS is designed and deployed as a cloud-hosted web application on Vercel's serverless infrastructure, addressing scalability, availability, and global accessibility requirements expected from a modern cloud application.

2. INTRODUCTION

RecipeOS is a utility-first, cloud-deployed cooking web application built with the philosophy of *"Cook Smarter, Not Harder."* It is a comprehensive recipe platform designed specifically for home cooks who want clean, ad-free, and interactive cooking guidance without the bloat of traditional recipe blogs.

The application is hosted on **Vercel's serverless cloud infrastructure**, making it globally accessible, highly available, and auto-scaling without the need for manual server management — a direct application of cloud computing principles covered in CSL605.

Key Highlights:

- **Interactive Gantt-style Timeline:** Parallel cooking tasks are visualized on a timeline, so users know exactly what to do and when — eliminating guesswork.
- **Cook Mode 2.0:** A full-screen, large-text, distraction-free interface with screen wake-lock and voice commands, built for messy hands.
- **Smart Ingredients:** Step-to-ingredient synchronization — hovering over a cooking step highlights the exact ingredients needed at that moment.
- **Goal-Based Cooking:** Recipes are filterable by health goals: Weight Loss, Muscle Gain, Heart Healthy, Plant-Based, Quick & Easy, and Family Meals.
- **Skill Progression:** A gamified XP and achievement system that levels up the user's cooking mastery across multiple skill categories.
- **Pantry Chef:** An AI-assisted feature that suggests recipes based on ingredients currently available in the user's pantry.
- **8 Recipe Categories:** Vegetarian, Non-Vegetarian, Breakfast, Snacks & Street Food, Main Course, Rice Dishes, Breads, and Sweets & Desserts.
- **16 Curated Indian Recipes** ranging from beginner (Masala Omelette, Poha) to advanced (Gulab Jamun, Vegetable Biryani, Aloo Paratha).

RecipeOS demonstrates the practical application of cloud computing concepts including serverless deployment, CDN-based static asset delivery, auto-scaling serverless functions, and cloud-based authentication — all key Lab Outcomes of CSL605.

3. SPECIFICATIONS / REQUIREMENTS

Software Requirements:

Component	Specification
Operating System	Windows 10/11, macOS, or Linux (Ubuntu 20.04+)
Node.js Runtime	Node.js v18.0 or higher
Package Manager	npm v9+ or yarn v1.22+
Framework	Next.js 14 (App Router)
CSS Framework	Tailwind CSS v3.4
Deployment Platform	AWS / Vercel (Serverless Cloud)
Version Control	Git + GitHub
Browser (Client)	Google Chrome / Firefox / Edge (latest)
Code Editor	Visual Studio Code
Authentication Service	Clerk (Cloud-based Auth)
Database / Backend	AWS

Hardware Requirements:

Component	Minimum	Recommended
Processor	Intel i3 / AMD Ryzen 3	Intel i5 / AMD Ryzen 5
RAM	4 GB	8 GB
Storage	10 GB free	20 GB SSD
Network	2 Mbps Internet	10 Mbps Broadband
Display	1280 × 720	1920 × 1080

4. TECHNOLOGY USED

1) Front End:

- **Next.js 14 (App Router):** React-based full-stack framework providing Server-Side Rendering (SSR), Static Site Generation (SSG), file-based routing, and API routes — the core frontend framework.
- **React.js 18:** Component-based UI library used for building reusable, stateful UI components like the recipe card, Cook Mode interface, and timeline.
- **Tailwind CSS v3.4:** Utility-first CSS framework for responsive, dark-themed design across all pages without writing custom CSS files.
- **Framer Motion:** Animation library used for smooth page transitions, interactive cooking step highlights, and micro-interactions.
- **Lucide React:** SVG icon library providing consistent, lightweight icons throughout the application interface.
- **Next/Image:** Optimized cloud image delivery component for automatic WebP conversion, lazy loading, and CDN caching of recipe images.

2) Back End / Cloud Infrastructure:

- **AWS:** API routes in Next.js are deployed as serverless functions on Vercel's global edge network — the primary cloud compute layer.
- **Vercel CDN & Edge Network:** Static assets (images, fonts, JS bundles) are distributed via Vercel's global CDN for low-latency delivery worldwide.
- **Clerk Authentication:** Cloud-based, managed authentication service providing sign-up, sign-in, session management, and user profile — deployed as a SaaS (Software as a Service) component.
- **Static JSON Data Store:** Recipe and category data managed as structured JSON files, served statically with zero database overhead — aligned with JAMstack cloud architecture.
- **Vercel Analytics:** Real-time cloud-based analytics tracking page performance, web vitals, and user engagement metrics.
- **GitHub:** Version control and source code hosting. Integrated with Vercel for Continuous Deployment (CD) — every push to main auto-deploys the application.

Cloud Computing Concepts Applied:

Concept	Implementation in RecipeOS
Serverless Computing (FaaS)	Vercel Serverless Functions for API routes
CDN / Edge Distribution	Recipe images & static assets via Vercel CDN

SaaS Integration	Clerk for cloud-hosted authentication
Auto-Scaling	Vercel auto-scales serverless functions on demand
CI/CD Pipeline	GitHub → Vercel automated deployment pipeline
High Availability	Vercel global edge with 99.99% uptime SLA

5. DESIGN

(Architecture / Flowchart / UML Diagrams)

A) System Architecture

RecipeOS follows a **Serverless JAMstack Cloud Architecture**. The client browser communicates with Vercel's Edge Network, which serves pre-rendered Next.js pages and routes API calls to serverless functions. Authentication is delegated to Clerk's SaaS cloud, and recipe data is stored as optimized static JSON files.

CLIENT LAYER	User's Browser → Next.js React App (SSR / SSG) Tailwind CSS Framer Motion Lucide Icons
VERCEL EDGE / CDN	Global CDN → Static Assets, Optimized Images Edge Functions → Routing, Middleware
SERVERLESS FUNCTIONS	Next.js API Routes → Recipe Data User Progress Auto-scaling FaaS on demand
SAAS SERVICES	Clerk Auth → Sign-Up, Sign-In, Session Mgmt Vercel Analytics → Performance Monitoring
DATA LAYER	Static JSON Files → Recipes, Categories, Goals Client-side localStorage → User Progress & XP
CI/CD PIPELINE	GitHub Repository → Auto-deploy to Vercel Branch Previews Instant Rollback

B) User Flow Diagram

START	User visits RecipeOS	▼
AUTH	Sign Up / Sign In via Clerk	▼
HOME	Landing Page: Browse by Category or Goal	▼
DISCOVER	Search / Filter Recipes by Category, Goal, or Ingredient	▼
DETAIL	Recipe Detail Page: Ingredients + Timeline View	▼
COOK	Enter Cook Mode 2.0 (Full-Screen, Timers, Voice)	▼
TRACK	Complete Recipe → Earn XP → Skill Progression	▼
END	View Progress Dashboard / Unlock Achievements	

6. IMPLEMENTATION

File / Folder	Purpose
app/page.tsx	Landing page — Hero, Features, Categories
app/recipes/page.tsx	All Recipes listing with category filters
app/recipes/[slug]/page.tsx	Dynamic recipe detail page with timeline
app/goal-based-cooking/	Goal-based recipe filter page
app/category/[cat]/	Category-specific recipe listing pages
app/sign-in / sign-up/	Clerk-powered authentication pages
app/your-progress/	User XP and skill progression dashboard
components/CookMode.tsx	Full-screen Cook Mode 2.0 component
components/Timeline.tsx	Gantt-style interactive cooking timeline
data/recipes/	Static JSON files for each recipe
public/images/	Optimized recipe images served via CDN
vercel.json	Vercel deployment and rewrite configuration

Implementation Steps:

Step 1 – Project Initialization: Created Next.js 14 project using *create-next-app* with TypeScript and App Router. Configured Tailwind CSS, ESLint, and path aliases.

Step 2 – Recipe Data Modeling: Designed JSON schema for recipes containing: slug, title, category, difficulty, xpReward, ingredients[], steps[], timeline[], and nutritionInfo.

Step 3 – Page Development: Built landing page, recipe listing, dynamic [slug] detail pages, category pages, and goal-based cooking page using Next.js App Router conventions.

Step 4 – Cook Mode 2.0: Implemented a full-screen cooking interface with step-by-step navigation, countdown timers, screen wake-lock API, and ingredient synchronization.

Step 5 – Timeline Component: Created an interactive Gantt-style cooking timeline where parallel tasks are displayed visually, with real-time progress tracking.

Step 6 – Authentication: Integrated Clerk for cloud-based sign-up/sign-in. Protected the progress dashboard and achievement routes using Clerk middleware.

Step 7 – Skill Progression: Built an XP and achievement system using client-side state and localStorage — tracking user mastery across categories

7. RESULT

RecipeOS was successfully designed, developed, and deployed as a cloud-hosted web application on Vercel. The application is live and accessible globally at:

Features Successfully Implemented:

Feature	Status	Details
Landing Page	✓ Working	Hero, Features, Categories, CTA
Recipe Listing Page	✓ Working	16 recipes with category filters
Recipe Detail Page	✓ Working	Ingredients, steps, timeline view
Cook Mode 2.0	✓ Working	Full-screen, timers, wake-lock
Interactive Timeline	✓ Working	Gantt-style parallel task view
Goal-Based Cooking	✓ Working	6 health goals, filtered recipes
8 Recipe Categories	✓ Working	Veg, Non-Veg, Breakfast, Snacks...
Skill Progression / XP	✓ Working	Beginner to Advanced, XP rewards
Pantry Chef	✓ Working	Ingredient-based recipe suggestion
User Authentication	✓ Working	Clerk sign-up/sign-in integrated
Cloud Deployment (Vercel)	✓ Live	Auto-scaling, CDN, CI/CD pipeline
Responsive Design	✓ Working	Mobile, tablet, and desktop

Outcome Summary:

The application successfully demonstrates the following Cloud Computing Lab Outcomes (LO1–LO6): virtualization and cloud service models (LO1), cloud deployment and infrastructure (LO2), SaaS integration and service-oriented architecture (LO3), auto-scaling and serverless computing (LO4), cloud security and authentication (LO5), and performance monitoring using cloud analytics (LO6).

Screenshot reference: All application screenshots are available at the live URL: <https://recipe-os-five.vercel.app/>

8. REFERENCES

[1] Next.js Official Documentation

<https://nextjs.org/docs>

Next.js App Router, Server Components, Routing, API Routes.

[2] Vercel Documentation

<https://vercel.com/docs>

Serverless Functions, Edge Network, CI/CD Deployment, Analytics.

[3] Tailwind CSS Documentation

<https://tailwindcss.com/docs>

Utility-first CSS classes, responsive design, dark mode.

[4] Clerk Authentication

<https://clerk.com/docs>

Cloud-based authentication, session management, middleware.

[5] React.js Documentation

<https://react.dev>

Component model, hooks, state management.

[6] Framer Motion

<https://www.framer.com/motion/>

React animation library for transitions and micro-interactions.

[7] MDN Web Docs – Screen Wake Lock API

https://developer.mozilla.org/en-US/docs/Web/API/Screen_Wake_Lock_API

Browser API used for Cook Mode hands-free screen management.

[8] NIST Definition of Cloud Computing

<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>

Foundational cloud computing model: SaaS, PaaS, IaaS definitions.

[9] GitHub – pal-adarsh/RecipeOS

<https://github.com/pal-adarsh/RecipeOS>

Source code repository for the RecipeOS project.
